Переход на аккумулятор **2S LiHV (8.7 В при полном заряде, 4.35 В/банка)** вместо стандартного LiPo (8.4 В) меняет рабочие режимы сразу в четырех узлах твоей схемы:

- **Делитель мониторинга АКБ (PA0, лист 01_Power_Input):** верхнее плечо 100 кОм, нижнее 47.5 кОм (коэффициент передачи $K = \frac{47.5}{147.5} \approx 0.32203$). При 8.70 В напряжение на пине PA0 составит $8.70 \times 0.32203 = \mathbf{2.802}$ В, что безопасно укладывается в динамический диапазон АЦП до 3.30 В (потолок делителя без клиппинга — 10.24 В, код АЦП при 8.70 В равен ≈ 3477).
- **Изолированное питание AMC1311 (лист 04_HV_Bus_Sense):** на схеме перед DC-DC изолятором B0505S-1WR3 (U4) стоит линейный предрегулятор LM78L05 (U5), питающийся от VBAT_SW. Ни в коем случае не исключай U5 при сборке (в BOM.md он пропущен в текстовом списке, хотя есть на схеме): B0505S-1WR3 имеет абсолютный максимум входа 6.0 В и от 8.7 В сгорит мгновенно. С установленным LM78L05 напряжение 8.7 В наоборот дает комфортный запас по падению напряжения (dropout ≈ 1.7–2.0 В).
- **Резонансный контур ZVS и TVS-супрессоры (M1/M2, лист 03_ZVS_PushPull):** в топологии Current-Fed Push-Pull пиковое напряжение на стоках равно $\pi \times V_{BAT}$. При 8.7 В идеальная амплитуда полуволны вырастет до 27.3 В, а с учетом добротности контура и паразитного звона — до **34–36 В**, что вплотную приближается к напряжению начала лавинного пробоя супрессоров 1.5SMC39A ($V_{WM} = 33.3$ В, $V_{BR} = 39$ В). На делителе детектора нуля (10 кОм/1 кОм, $K = 1/11$) это даст 3.27 В на входах PA1/PA7 — на самой грани открытия защитной сборки BAT54C.
- **Микросоленоид 0520D (SOL1, лист 06_Pneumatics):** катушка на 6 В подключена напрямую к VBAT_SW. При 8.7 В рассеиваемая на обмотке мощность вырастает в $(8.7/6.0)^2 = \mathbf{2.1}$ раза относительно номинала. Для предпродувки 150 мс и коротких очередей это даже плюс (клапан открывается резче), но при удержании курка до отсечки 15 с соленоид будет сильно греться.

Архитектурный нюанс Step 0: вывод PA8 (MCO) и драйвер SiC UCC27511

По схеме (02_MCU_Control_UI и 05_SiC_Nanosecond_Stage) вывод PA8 через резистор R47 (110 Ом) заведен напрямую на вход IN+ драйвера UCC27511, управляющего затвором C3M0075120K.

- Если плата WeAct тестируется отдельно на столе или повышающий модуль +15V_SIC (MT3608) физически отключен, вывод меандра MCO на PA8 полностью безопасен.
- Если же STEP 0 выполняется на собранной плате с поданными вторичными питаниями (+15.0 В на UCC27511), подача непрерывной частоты 10.625 МГц с MCO на PA8 заставит драйвер непрерывно перезаряжать входную емкость

затвора SiC-транзистора ($P = C_{iss} V_{GS}^2 f$), что за секунды перегреет затворный резистор R42 (4.7 Ом, 0603) и сам драйвер UCC27511.

Поэтому в bsp_gpio.h выход MCO на PA8 вынесен под явный макрос компиляции BSP_STEP0_ENABLE_MCO_PA8, а силовые затворные линии (PA8, PA10, PA11, PB0) по умолчанию на этапе Step 0 удерживаются в безопасном состоянии.

1. Заголовочный файл тактирования: Core/Inc/bsp_clock.h

C

```
#ifndef BSP_CLOCK_H
#define BSP_CLOCK_H

#include "stm32g4xx.h"
#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

/** Системные константы тактирования */
#define BSP_SYSCLK_HZ               (170000000UL)
#define BSP_SYSCLK_MHZ              (170UL)
#define BSP_SYSTICK_FREQ_HZ         (1000UL)

/** Иерархия приоритетов NVIC (4 бита: 0..15) */
#define BSP_NVIC_PRIO_HRTIM_FLT     (0UL)  /* Аппаратный OVP > 900 В */
#define BSP_NVIC_PRIO_COMP_HV       (1UL)  /* Отсечка накачки 780 В */
#define BSP_NVIC_PRIO_HRTIM_MST     (2UL)  /* Синхронный останов ZVS в нуле */
#define BSP_NVIC_PRIO_EXTI_FIRE     (4UL)  /* Курок KEY_FIRE (PB3) */
#define BSP_NVIC_PRIO_EXTI_ENC      (5UL)  /* Кнопка энкодера (PB2) */
#define BSP_NVIC_PRIO_SYSTICK       (6UL)  /* Системный квант 1 мс и телеметрия */

/** Пределы счетчиков аварийного выхода при ожидании аппаратных флагов */
#define BSP_CLK_TIMEOUT_HSI         (50000UL)
#define BSP_CLK_TIMEOUT_PWR         (50000UL)
#define BSP_CLK_TIMEOUT_FLASH       (50000UL)
#define BSP_CLK_TIMEOUT_PLL         (100000UL)
#define BSP_CLK_TIMEOUT_SWS         (100000UL)

/** Коды возврата инициализации тактирования */
typedef enum {
    BSP_CLK_OK = 0,
    BSP_CLK_ERR_PWR_VOS_TIMEOUT,
    BSP_CLK_ERR_FLASH_LATENCY_TIMEOUT,
    BSP_CLK_ERR_HSI_TIMEOUT,
    BSP_CLK_ERR_PLL_STOP_TIMEOUT,
    BSP_CLK_ERR_PLL_LOCK_TIMEOUT,
    BSP_CLK_ERR_SWS_SWITCH_TIMEOUT
} bsp_clk_status_t;

/**
 * @brief Разгон ядра Cortex-M4F до 170 МГц от HSI16 (Range 1 Boost Mode)
 *        и активация аппаратного блока с плавающей точкой (FPU).
 * @return BSP_CLK_OK при успешном захвате PLL и переключении шин.
 */
bsp_clk_status_t bsp_clock_init_170mhz(void);
```

```
/**
 * @brief Инициализация системного таймера SysTick на период 1.0 мс (170 000 тактов).
 */
void bsp_systick_init(void);

/**
 * @brief Неблокирующее чтение 64-битного системного счетчика миллисекунд.
 *        Реализовано через lock-free чтение без использования __disable_irq().
 * @return Время с момента старта в миллисекундах.
 */
uint64_t systick_millis(void);

/**
 * @brief Прецизионная микросекундная задержка через аппаратный регистр SysTick->VAL.
 * @param us Длительность задержки в микросекундах.
 */
void delay_us(uint32_t us);

#ifdef __cplusplus
}
#endif


#endif /* BSP_CLOCK_H */
```

2. Реализация тактирования и SysTick: Core/Src/bsp_clock.c

C

```
#include "bsp_clock.h"

/* Внутренний 64-битный счетчик миллисекунд, разделенный на два 32-битных слова
 * для гарантированно атомарного lock-free чтения на 32-битном ядре Cortex-M4F */
static volatile uint32_t g_systick_ms_low  = 0UL;
static volatile uint32_t g_systick_ms_high = 0UL;

/**
 * @brief Включение сопроцессора FPU (CP10 и CP11 Full Access).
 */
static inline void bsp_fpu_enable(void)
{
    /* CPACR[23:20] = 1111b: полный доступ к регистрам S0..S31 */
    SCB->CPACR |= ((3UL << (10UL * 2UL)) | (3UL << (11UL * 2UL)));
    __DSB();
    __ISB();
}

bsp_clk_status_t bsp_clock_init_170mhz(void)
{
    uint32_t timeout;

    /* 0. Активация FPU до выполнения любых математических операций */
    bsp_fpu_enable();
```

```
    /* 1. Включение тактирования контроллера питания PWR на шине APB1
*/
    RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
    __DSB();

    /* 2. Установка основного регулятора ядра в Voltage Scaling Range
1 (VOS = 01b) */
    PWR->CR1 = (PWR->CR1 & ~PWR_CR1_VOS_Msk) | (1UL <<
PWR_CR1_VOS_Pos);
    __DSB();

    timeout = BSP_CLK_TIMEOUT_PWR;
    while ((PWR->SR2 & PWR_SR2_VOSF) != 0UL)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_PWR_VOS_TIMEOUT;
        }
    }

    /* 3. Перевод регулятора напряжения в Range 1 Boost Mode (R1MODE =
0)
     * Обязателен в STM32G4 для частот SYSCLK > 150 МГц (RM0440 п.
6.1.5) */
    PWR->CR5 &= ~PWR_CR5_R1MODE;
    __DSB();

    /* 4. Конфигурация контроллера Flash-памяти под 170 МГц в Boost
Mode:
     *    - LATENCY = 4 WS (4 такта ожидания, RM0440 Table 19)
     *    - ICEN = 1 (кэш инструкций)
     *    - DCEN = 1 (кэш данных)
     *    - PRFTEN = 1 (упреждающая выборка Prefetch Buffer) */
    FLASH->ACR = FLASH_ACR_LATENCY_4WS |
                 FLASH_ACR_ICEN        |
                 FLASH_ACR_DCEN        |
                 FLASH_ACR_PRFTEN;
    __DSB();
    __ISB();

    /* Контрольное чтение регистра FLASH->ACR с таймаутом */
    timeout = BSP_CLK_TIMEOUT_FLASH;
    while ((FLASH->ACR & FLASH_ACR_LATENCY_Msk) !=
FLASH_ACR_LATENCY_4WS)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_FLASH_LATENCY_TIMEOUT;
        }
    }
```

```
    /* 5. Включение внутреннего генератора HSI16 (16 МГц) и ожидание
стабилизации */
    RCC->CR |= RCC_CR_HSION;
    __DSB();

    timeout = BSP_CLK_TIMEOUT_HSI;
    while ((RCC->CR & RCC_CR_HSIRDY) == 0UL)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_HSI_TIMEOUT;
        }
    }

    /* 6. Гарантированное отключение PLL перед модификацией делителей
*/
    RCC->CR &= ~RCC_CR_PLLON;
    __DSB();

    timeout = BSP_CLK_TIMEOUT_PLL;
    while ((RCC->CR & RCC_CR_PLLRDY) != 0UL)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_PLL_STOP_TIMEOUT;
        }
    }

    /* 7. Настройка главной ФАПЧ (Main PLL):
     *    - Источник: HSI16 = 16 МГц (PLLSRC = 10b)
     *    - Делитель PLLM = /4 -> кодируется значением (4 - 1) = 3
(0011b).
     *      Частота сравнения ФАПЧ: 16 МГц / 4 = 4.0 МГц.
     *    - Множитель PLLN = *85 -> (85UL << RCC_PLLCFGR_PLLN_Pos).
     *      Частота ГУН (VCO): 4.0 МГц * 85 = 340.0 МГц.
     *    - Выходной делитель PLLR = /2 -> кодируется 00b (0UL <<
RCC_PLLCFGR_PLLR_Pos).
     *      Итоговая частота ядра: 340.0 МГц / 2 = 170.0 МГц.
     *    - Разрешение выхода домена R: PLLREN = 1. */
    RCC->PLLCFGR = (2UL  << RCC_PLLCFGR_PLLSRC_Pos) | /* 10b: HSI16 */
                   (3UL  << RCC_PLLCFGR_PLLM_Pos)   | /* 0011b: PLLM =
/4 */
                   (85UL << RCC_PLLCFGR_PLLN_Pos)   | /* PLLN = *85 */
                   (0UL  << RCC_PLLCFGR_PLLR_Pos)   | /* 00b: PLLR =
/2 */
                   RCC_PLLCFGR_PLLREN;                /* Включить
выход PLLCLK */
    __DSB();

    /* 8. Запуск PLL и ожидание захвата частоты (PLLRDY) */
    RCC->CR |= RCC_CR_PLLON;
```

```
    __DSB();

    timeout = BSP_CLK_TIMEOUT_PLL;
    while ((RCC->CR & RCC_CR_PLLRDY) == 0UL)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_PLL_LOCK_TIMEOUT;
        }
    }

    /* 9. Ступенчатый переход шины AHB (требование RM0440 п. 6.1.5 при
разгоне > 80 МГц):
     *    Сначала выставляем предделитель HPRE = /2 (85 МГц),
переключаем SW на PLL,
     *    выдерживаем паузу > 1 мкс для стабилизации токов ядра и
переводим HPRE = /1 (170 МГц). */
    RCC->CFGR = (RCC->CFGR & ~(RCC_CFGR_HPRE_Msk | RCC_CFGR_PPRE1_Msk
| RCC_CFGR_PPRE2_Msk)) |
                RCC_CFGR_HPRE_DIV2  |
                RCC_CFGR_PPRE1_DIV1 |
                RCC_CFGR_PPRE2_DIV1;
    __DSB();

    /* Переключение мультиплексора SYSCLK на PLL (SW = 11b) */
    RCC->CFGR = (RCC->CFGR & ~RCC_CFGR_SW_Msk) | RCC_CFGR_SW_PLL;
    __DSB();
    __ISB();

    timeout = BSP_CLK_TIMEOUT_SWS;
    while ((RCC->CFGR & RCC_CFGR_SWS_Msk) != RCC_CFGR_SWS_PLL)
    {
        if (--timeout == 0UL)
        {
            return BSP_CLK_ERR_SWS_SWITCH_TIMEOUT;
        }
    }

    /* Пауза ~2 мкс на частоте 85 МГц для завершения переходного
процесса LDO ядра */
    for (volatile uint32_t i = 0UL; i < 200UL; i++)
    {
        __NOP();
    }

    /* Выход на полную частоту шин: AHB = 170 МГц (DIV1), APB1 = 170
МГц, APB2 = 170 МГц */
    RCC->CFGR &= ~RCC_CFGR_HPRE_Msk; /* 0000b: HCLK = SYSCLK / 1 = 170
МГц */
    __DSB();
    __ISB();
```

```
    return BSP_CLK_OK;
}

void bsp_systick_init(void)
{
    /* Останов таймера перед конфигурацией */
    SysTick->CTRL = 0UL;

    /* Период ровно 1.0 мс при тактовой 170 МГц: (170 000 000 / 1000)
- 1 = 169 999 */
    SysTick->LOAD = (BSP_SYSCLK_HZ / BSP_SYSTICK_FREQ_HZ) - 1UL;
    SysTick->VAL  = 0UL;

    /* Установка низкого приоритета (6 из 15), чтобы SysTick никогда
не задерживал
     * прерывания аварий HRTIM (приоритет 0) и компараторов ZVS/HV
(приоритет 1..2) */
    NVIC_SetPriority(SysTick_IRQn, BSP_NVIC_PRIO_SYSTICK);

    /* Запуск SysTick: тактирование от ядра (CLKSOURCE = 1),
прерывание (TICKINT = 1), ENABLE = 1 */
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
    __DSB();
    __ISB();
}

uint64_t systick_millis(void)
{
    uint32_t hi1;
    uint32_t lo;
    uint32_t hi2;
    uint32_t guard = 10UL;

    /* Lock-free чтение 64-битного значения без блокировки прерываний
(__disable_irq запрещен):
     * если между чтением старшего и младшего слова произошло
прерывание с переносом разряда,
     * цикл повторит чтение (максимум 1 дополнительная итерация). */
    do
    {
        hi1 = g_systick_ms_high;
        lo  = g_systick_ms_low;
        hi2 = g_systick_ms_high;
    } while ((hi1 != hi2) && (--guard > 0UL));

    return (((uint64_t)hi2 << 32U) | (uint64_t)lo);
}
```

```
void delay_us(uint32_t us)
{
    if (us == 0UL)
    {
        return;
    }

    /* На частоте 170 МГц 1 мкс соответствует ровно 170 тактам SysTick
*/
    const uint32_t reload = SysTick->LOAD + 1UL;
    uint64_t total_ticks  = (uint64_t)us * (uint64_t)BSP_SYSCLK_MHZ;
    uint64_t elapsed      = 0ULL;
    uint32_t last_val     = SysTick->VAL;

    /* Аварийный ограничитель итераций против зависания при
остановленном SysTick */
    uint32_t max_loops    = (us * 256UL) + 1000UL;

    while ((elapsed < total_ticks) && (--max_loops > 0UL))
    {
        uint32_t now_val = SysTick->VAL;
        if (now_val != last_val)
        {
            if (now_val < last_val)
            {
                /* Счетчик SysTick считает вниз: обычный шаг внутри
миллисекунды */
                elapsed += (uint64_t)(last_val - now_val);
            }
            else
            {
                /* Переход через 0 и перезагрузка из регистра LOAD */
                elapsed += (uint64_t)( reload - now_val + last_val );
            }
            last_val = now_val;
        }
    }
}

/**
 * @brief Обработчик прерывания системного таймера (1.0 мс).
 */
void SysTick_Handler(void)
{
    uint32_t next_low = g_systick_ms_low + 1UL;
    g_systick_ms_low  = next_low;
    if (next_low == 0UL)
    {
        g_systick_ms_high++;
    }
}
```

3. Заголовочный файл GPIO и ЭМС-защиты: Bsp/Gpio/bsp_gpio.h

C

```
#ifndef BSP_GPIO_H
#define BSP_GPIO_H

#include "stm32g4xx.h"
#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

/**
 * @brief Флаг вывода частоты SYSCLK/16 (10.625 МГц) на пин PA8 (MCO).
 * ВНИМАНИЕ: Устанавливать в 1 ТОЛЬКО при отключенном питании +15V драйвера UCC27511
 * или на нераспаянной плате WeAct! При собранной силовой части держать строго 0.
 */
#ifndef BSP_STEP0_ENABLE_MCO_PA8
#define BSP_STEP0_ENABLE_MCO_PA8    (0)
#endif

/**
 * @brief Аппаратная ЭМС-инициализация выводов для Step 0 (Cold Board Verification):
 *        - переводит все неиспользуемые пины в Analog Mode (MODER = 11b), No-Pull
(PUPDR = 00b)
 *          для отключения входных триггеров Шмитта при dV/dt > 50 В/нс;
 *        - жестко сохраняет PA13 (SWDIO) и PA14 (SWCLK) в AF0 с штатными подтяжками;
 *        - настраивает индикаторный светодиод платы WeAct (PC13) как Output Push-
Pull Low Speed.
 */
void bsp_gpio_emc_init(void);

/**
 * @brief Опциональный вывод системной частоты через предделитель MCO (/16 = 10.625
МГц) на PA8.
 */
void bsp_gpio_mco_pa8_enable(void);

/** Атомарное управление светодиодом PC13 через регистр BSRR */
static inline void bsp_led_pc13_on(void)
{
    GPIOC->BSRR = GPIO_BSRR_BS13;
}

static inline void bsp_led_pc13_off(void)
{
    GPIOC->BSRR = GPIO_BSRR_BR13;
}

static inline void bsp_led_pc13_toggle(void)
{
    /* Чтение ODR и атомарная запись в BSRR без неатомарного RMW над ODR */
    if ((GPIOC->ODR & GPIO_ODR_OD13) != 0UL)
    {
        GPIOC->BSRR = GPIO_BSRR_BR13;
    }
    else
    {
```

```
        GPIOC->BSRR = GPIO_BSRR_BS13;
    }
}

#ifdef __cplusplus
}
#endif

#endif /* BSP_GPIO_H */
```

4. Реализация ЭМС-защиты GPIO: Bsp/Gpio/bsp_gpio.c

Обрати внимание на важную кремниевую деталь STM32G4 (RM0440 п. 9.4.1): после аппаратного сброса выводы JTAG (PA15 JTDI, PB3 JTDO, PB4 NJTRST) остаются в режиме Alternate Function (10b), а не Analog (11b). Если просто обнулить PUPDR, оставив MODER по умолчанию, их входные триггеры Шмитта останутся открытыми для емкостных наводок от разрядника. Код ниже принудительно закрывает в Analog Mode (11b) абсолютно все линии портов A, B, C, оставляя в AF0 исключительно PA13 и PA14.

C

```
#include "bsp_gpio.h"

void bsp_gpio_emc_init(void)
{
    /* 1. Включение тактирования портов GPIOA, GPIOB, GPIOC на шине AHB2 */
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN |
                    RCC_AHB2ENR_GPIOBEN |
                    RCC_AHB2ENR_GPIOCEN;
    __DSB();

    /* ============================================================================
     * ПОРТ GPIOA:
     * Все выводы PA0..PA12 и PA15 -> Analog Mode (11b), No-Pull (00b).
     * ИСКЛЮЧЕНИЕ: PA13 (SWDIO) и PA14 (SWCLK) строго остаются в AF0 (MODER = 10b):
     *   - PA13 (биты 27:26): MODER = 10b (AF), PUPDR = 01b (Pull-Up по спецификации
SWD)
     *   - PA14 (биты 29:28): MODER = 10b (AF), PUPDR = 10b (Pull-Down по
спецификации SWD)
     * ============================================================================ */
    GPIOA->AFR[1] &= ~((0xFUL << ((13UL - 8UL) * 4UL)) |
                       (0xFUL << ((14UL - 8UL) * 4UL))); /* AF0 для PA13, PA14 */

    /* Маска MODER для GPIOA: все пины 11b, кроме PA13 (10b) и PA14 (10b) ->
0xEBFFFFFF */
    GPIOA->MODER = (0xFFFFFFFFUL & ~((3UL << (13UL * 2UL)) | (3UL << (14UL * 2UL))))
|
                   ((2UL << (13UL * 2UL)) | (2UL << (14UL * 2UL)));

    /* Маска PUPDR для GPIOA: все пины 00b (No-Pull), PA13 = 01b (PU), PA14 = 10b
(PD) */
    GPIOA->PUPDR = (1UL << (13UL * 2UL)) | (2UL << (14UL * 2UL));

    /* ============================================================================
     * ПОРТ GPIOB:
     * Все выводы PB0..PB15 (включая бывшие JTAG-выводы PB3/JTDO и PB4/NJTRST)
     * принудительно переводятся в Analog Mode (11b), No-Pull (00b).
     * На линии PB0 (ключ клапана AO3400A) на плате распаян внешний стягивающий
     * резистор R46 = 10 кОм на GND, который жестко держит затвор закрытым.
     * ============================================================================ */
```

```
    GPIOB->MODER = 0xFFFFFFFFUL;
    GPIOB->PUPDR = 0x00000000UL;

    /* ============================================================================
     * ПОРТ GPIOC:
     * Все выводы кроме PC13 -> Analog Mode (11b), No-Pull (00b).
     * ИСКЛЮЧЕНИЕ: PC13 (светодиод платы WeAct Studio):
     *   - Выставляем начальный низкий уровень через BSRR до включения драйвера пина
     *   - MODER13   = 01b (General Purpose Output)
     *   - OTYPER13  = 0b  (Push-Pull)
     *   - OSPEEDR13 = 00b (Low Speed — минимизация dI/dt помех от самого индикатора)
     *   - PUPDR13   = 00b (No-Pull)
     * ============================================================================ */
    GPIOC->BSRR    = GPIO_BSRR_BR13;
    GPIOC->OTYPER &= ~GPIO_OTYPER_OT13;
    GPIOC->OSPEEDR &= ~(3UL << (13UL * 2UL));
    GPIOC->PUPDR   = 0x00000000UL;
    GPIOC->MODER   = (0xFFFFFFFFUL & ~(3UL << (13UL * 2UL))) | (1UL << (13UL * 2UL));

#if (BSP_STEP0_ENABLE_MCO_PA8 == 1)
    bsp_gpio_mco_pa8_enable();
#endif

    __DSB();
    __ISB();
}

void bsp_gpio_mco_pa8_enable(void)
{
    /* 1. Настройка предделителя и источника MCO в RCC->CFGR:
     *    - MCOPRE = 100b (деление на 16): 170 МГц / 16 = 10.625 МГц
     *      (в полосе пропускания осциллографа Siglent SDS 1202CNL 200 МГц даст
четкий фронт)
     *    - MCOSEL = 0001b (SYSCLK) */
    RCC->CFGR = (RCC->CFGR & ~(RCC_CFGR_MCOSEL_Msk | RCC_CFGR_MCOPRE_Msk)) |
                (4UL << RCC_CFGR_MCOPRE_Pos) | /* 100b: MCO / 16 */
                (1UL << RCC_CFGR_MCOSEL_Pos);  /* 0001b: SYSCLK */
    __DSB();

    /* 2. Конфигурация вывода PA8 под MCO (AF0, Very High Speed, Push-Pull, No-Pull)
*/
    GPIOA->OTYPER  &= ~GPIO_OTYPER_OT8;
    GPIOA->OSPEEDR |= (3UL << (8UL * 2UL));             /* 11b: Very High Speed */
    GPIOA->PUPDR   &= ~(3UL << (8UL * 2UL));            /* 00b: No-Pull */
    GPIOA->AFR[1]  &= ~(0xFUL << ((8UL - 8UL) * 4UL));  /* AF0: MCO */
    GPIOA->MODER    = (GPIOA->MODER & ~(3UL << (8UL * 2UL))) |
                      (2UL << (8UL * 2UL));             /* 10b: Alternate Function
*/
    __DSB();
}
```

5. Главный модуль верификации Step 0: Core/Src/main.c

Если на этапе инициализации тактирования сработает аварийный таймаут (например, просадка по шине питания ядра), контроллер перейдет в безопасный диагностический режим с быстрой вспышкой светодиода PC13 каждые ~100 мс, сигнализируя об ошибке захвата PLL. При штатном выходе на 170 МГц PC13 переключается строго каждые 500 мс (период меандра 1.000 с, частота 1.000 Гц).

C

```
#include "stm32g4xx.h"
#include "bsp_clock.h"
#include "bsp_gpio.h"

int main(void)
{
    /* 1. Первичная ЭМС-защита: глушим входные буферы Шмитта всех свободных пинов
     *    сразу после старта, сохраняя интерфейс отладки SWD (PA13/PA14). */
    bsp_gpio_emc_init();

    /* 2. Разгон ядра до 170 МГц (HSI16 -> PLL 340/2 МГц, Boost Mode Range 1, 4 WS)
*/
    bsp_clk_status_t clk_status = bsp_clock_init_170mhz();

    if (clk_status != BSP_CLK_OK)
    {
        /* Аварийная индикация отказа PLL (быстрое мигание на базовой частоте HSI16):
         * не блокирует SWD и сразу видна визуально. */
        while (1)
        {
            bsp_led_pc13_toggle();
            for (volatile uint32_t i = 0UL; i < 160000UL; i++)
            {
                __NOP();
            }
        }
    }

    /* 3. Запуск системного таймера SysTick (квант 1.0 мс, приоритет NVIC = 6) */
    bsp_systick_init();

    /* Короткий стартовый строб 50 мс через delay_us() для проверки микросекундного
таймера */
    bsp_led_pc13_on();
    delay_us(50000UL);
    bsp_led_pc13_off();

    uint64_t last_toggle_ms = systick_millis();

    /* 4. Основной цикл верификации Step 0 */
    while (1)
    {
        uint64_t now_ms = systick_millis();

        /* Инверсия контрольного светодиода PC13 ровно каждые 500 мс */
        if ((now_ms - last_toggle_ms) >= 500ULL)
        {
            last_toggle_ms += 500ULL;
            bsp_led_pc13_toggle();
        }
    }
}
```

Протокол приборной верификации Step 0 (Мультиметр + Siglent SDS 1202CNL)

Перед подачей питания убедись, что предохранитель F1 (20 А) или лабораторный БП подключен с ограничением тока 0.25 А (так как силовая накачка ZVS и клапан на STEP 0 закрыты, общее потребление платы должно составлять около 35–55 мА с учетом балласта R21 = 300 Ом на выходе B0505S-1WR3).

1. Контроль вторичных шин питания и делителя АКБ (Мультиметр)

1. **Входная шина VBAT_SW (после P-MOSFET AOD4185 при включенном тумблере SW_PWR):**
    - Замер между полигоном GND и стоком Q1 (AOD4185). Напряжение должно совпадать с напряжением аккумулятора (8.70 В на полном заряде LiHV) без падения более 5 мВ.
2. **Пин мониторинга батареи PA0 (NetC1_1 на C1** 100 нФ**):**
    - При входном напряжении 8.70 В мультиметр на конденсаторе C1 должен показать $\mathbf{2.80}$ В $\pm$ $\mathbf{0.03}$ В. Поскольку PA0 переведен в Analog Mode (11b) без подтяжек (PUPDR = 00b), входной импеданс пина бесконечен и не нагружает делитель 100 кОм/47.5 кОм.
3. **Системная шина +3.3V (выход модуля MP2315):**
    - Замер на керамике C31/C32 возле выводов питания WeAct STM32G474CEU6: норма $\mathbf{3.30}$ В $\pm$ $\mathbf{0.05}$ В.
4. **Шина драйвера SiC +15V_SIC (выход модуля MT3608):**
    - Замер на конденсаторе C25 (10 мкФ) / выводе VCC (pin 1) UCC27511 относительно SIC_KELVIN_GND: норма $\mathbf{15.0}$ В $\pm$ $\mathbf{0.2}$ В.
5. **Изолированная шина горячей стороны +5.0V ISO (AMC1311):**
    - **Важно:** черный щуп мультиметра ставится строго на изолированный полигон **HV_GND** (вывод 4 GND1 микросхемы AMC1311 или минусовой вывод C19/C20), а не на цифровую землю!
    - Красный щуп — на конденсатор C17 (100 нФ, вывод 1 VDD1 AMC1311). Норма после стабилизатора ME6211C50M5G: $\mathbf{5.00}$ В $\pm$ $\mathbf{0.05}$ В.

2. Проверка отсутствия паразитных потенциалов на силовых затворах

Не снимая питания, проверь мультиметром в режиме DCV напряжения на входах силовых ключей:

- Затвор ключа клапана Q5 (AO3400A, соленоид): строго 0.00 В (прижат резистором R46 = 10 кОм).
- Выходы OUTA (pin 7) и OUTB (pin 5) драйвера UCC27524DR (затворы M1, M2): строго 0.00 В.
- Затвор Q4 (C3M0075120K) относительно истока Кельвина (pin 3): строго 0.00 В.

3. Инструментальная верификация тактирования 170 МГц (Осциллограф Siglent SDS 1202CNL)

1. **Проверка кванта SysTick на выводе PC13 (безопасный метод на полностью собранной плате):**
    - Подключи щуп PP215 в режиме 10:1 к выводу PC13 платы WeAct.
    - Включи аппаратный частотомер осциллографа (Measure -> Frequency, Period, +Width).
    - **Критерий успеха:** период сигнала равен ровно $\mathbf{1.000}$ с, длительность положительного импульса $\mathbf{500.0}$ мс, частота $\mathbf{1.000}$ Гц. Если бы PLL не запустилась и ядро осталось на HSI16 (16 МГц), при регистре SysTick->LOAD = 169999 длительность полупериода растянулась бы до 500 мс $\times \frac{170}{16} = \mathbf{5.3125}$ с.
2. **Прямой замер частоты ядра через MCO на выводе PA8 (только при обесточенном +15V_SIC или до запайки R47):**
    - Собери проект с #define BSP_STEP0_ENABLE_MCO_PA8 (1).
    - Встань щупом 10:1 с короткой пружинкой заземления на вывод PA8.
    - **Критерий успеха:** на экране Siglent SDS 1202CNL стабильный меандр амплитудой 3.3 В с частотой ровно:

$$f_{\text{MCO}} = \frac{170.000\ \text{МГц}}{16} = \mathbf{10.625}\ \text{МГц} \quad (T = 94.12\ \text{нс})$$

    - После проверки верни BSP_STEP0_ENABLE_MCO_PA8 в 0 перед переходом к STEP 1 (Valve Integrity Test) и STEP 2 (Dry Run Gates Check).